uc3m-thesis-ieee

Package Documentation — v0.4.0

Universidad Carlos III de Madrid thesis template, IEEE style

---

1. Configuration

The template exports a single `conf` function, which you apply to your document with `#show: conf.with(...)`.

1.1. Minimal example

`codly`

```typ
#import "@preview/uc3m-thesis-ieee:0.4.0": conf

#show: conf.with(
  title: "My Bachelor Thesis",
  author: "GUL UC3M",
  degree: "Computer Science and Engineering",
  advisors: ("Prof. L",),
  location: "Leganés, Madrid",
  thesis-type: "TFG",
  date: datetime(year: 2025, month: 6, day: 15),
  language: "en",
  abstract: (
    body: [A short description of my thesis.],
    keywords: ("Keyword 1", "Keyword 2", "Keyword 3"),
  ),
  genai-declaration: (usage: false),
)

= Introduction

My thesis starts here.
```

1.2. Abstract

The `abstract` parameter takes a dictionary with the following keys:

Key	Type	Description
`body`*	`content`	The abstract text.
`keywords`*	`array of str`	Between 2 and 5 keywords. See IEEE Taxonomy.

```typ
  abstract: (
  body: [
    This thesis presents an implementation of interrupts and timers
```

```
    in the CREATOR simulator.
  ],
  keywords: ("Operating Systems", "Interrupts", "RISC-V", "Simulation"),
),
```

For Spanish documents, it is required to provide an `english-abstract` with the same structure:

```typ
language: "es",
abstract: (
  body: [Una breve descripción de mi tesis.],
  keywords: ("Sistemas Operativos", "Interrupciones", "RISC-V"),
),
english-abstract: (
  body: [A brief description of my thesis.],
  keywords: ("Operating Systems", "Interrupts", "RISC-V"),
),
```

1.3. Epigraph

An optional opening quote placed before the abstract. Provide a dictionary with:

Key	Type	Description
`quote`*	`content`	The quote text.
`author`*	`str`	The author of the quote.
`source`	`str`	(Optional) The source where the quote was found.

```typ
epigraph: (
  quote: [The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise.],
  author: "Edsger W. Dijkstra",
  source: "The Humble Programmer, 1972",
),
```

1.4. Outlines

The `outlines` parameter accepts an optional dictionary with the following boolean keys. All keys are optional (default `false`):

Key	Type	Description
`figures`	`bool`	Include a list of figures.
`tables`	`bool`	Include a list of tables.
`listings`	`bool`	Include a list of code listings.
`custom`	`array of content`	Additional custom outlines (e.g. algorithms).

```typ
 outlines: (
   figures: true,
   tables: true,
   listings: false,
   // custom: (
```

```
  //   outline(
  //     title: [List of Algorithms],
  //     target: figure.where(kind: "algorithm"),
  //   ),
  // ),
),
```

1.5. Abbreviations

You can pass abbreviations as a dictionary mapping abbreviations to their expansions (automatically sorted alphabetically):

```typ
abbreviations: (
  API: "Application Programming Interface",
  CPU: "Central Processing Unit",
  ML: "Machine Learning",
  TFG: "Trabajo de Fin de Grado",
),
```

Or as custom `content` if you need full control over the layout:

```typ
abbreviations: [
  #table(
    columns: (auto, 1fr),
    [*API*], [Application Programming Interface],
    [*ML*],  [Machine Learning],
  )
],
```

1.6. Glossary

The template integrates with `glossarium` for automatic glossary and acronym management.

Define your entries in a separate file (e.g. `config/glossary.typ`):

```typ
#let glossary-entries = (
  (
    key: "API",
    short: "API",
    long: "Application Programming Interface",
    description: "A set of protocols and tools for building software.",
  ),
  (
    key: "ML",
    short: "ML",
    long: "Machine Learning",
    description: "A subset of AI that enables computers to learn from data.",
  ),
)
```

Then import it in your main file and pass it to `conf`:

```typ
#import "config/glossary.typ": glossary-entries

#show: conf.with(
  // ...
  glossary: glossary-entries,
)
```

To reference a glossary term in the body text, use `#gls("key")` (singular) or `#glspl("key")` (plural). These are re-exported from `glossarium` and available after the `#show: conf.with(...)` call:

```typ
#import "@preview/uc3m-thesis-ieee:0.4.0": conf
#import "@preview/glossarium:0.5.9": gls, glspl

// In the body:
The #gls("API") exposes several endpoints.
Multiple #glspl("ML") models were evaluated.
```

Alternatively, you can pass fully custom `content` to `glossary` if you don't want to use `glossarium`.

1.7. Generative AI Declaration

The `genai-declaration` parameter is **required** by the university. You can either:

1.7.1. Option A: Use the built-in template (recommended)

If you did not use generative AI (`usage: false`):

```typ
genai-declaration: (usage: false),
```

If you did use generative AI (`usage: true`), it is required to provide the full declaration:

Pass a dictionary with the following keys:

Key	Type	Required	Description
`usage`*	`bool`	Always	Whether you used generative AI in the thesis.
`data-usage`	`dictionary`	If `usage` is `true`	Data usage declaration. See below.
`technical-usage`	`dictionary`	If `usage` is `true`	Technical usage declaration. See below.
`usage-reflection`	`content`	If `usage` is `true`	Personal reflection on AI usage.

`data-usage` keys — all are `bool` (`true` = YES, `false` = NO):

Key	Description
`confidential`	Did you submit confidential data (trade secrets, proprietary info)?
`copyright`	Did you submit copyrighted materials?
`personal`	Did you submit personal data?
`tos`	Did your usage respect the tool's terms of use?

`technical-usage` keys — include only those that apply, with a brief description of the prompt and interaction. Items are automatically grouped into two sections matching the official form:

Documentation and drafting: `reflection`, `review`, `research`, `references`, `summary`, `translation`

Develop specific content: `assistance-coding`, `generating-content`, `optimization`, `data-processing`, `idea-inspiration`, `other`

An optional `tool` key (`str`) declares the AI system name and version (e.g. `"ChatGPT 4o"`).

```typ
genai-declaration: (
  usage: true,
  data-usage: (
    // true = YES / used with authorization; false = NO / not used
    confidential: false,
    copyright: false,
    personal: false,
    tos: true,
  ),
  technical-usage: (
    // Name of the AI system and version (optional):
    // tool: "ChatGPT 4o",

    // ── Documentation and drafting ──────────────────────────────
    reflection: [I asked for a list of alternatives to address the
      problem stated at the beginning of the paper.],
    // review: [...],
    // research: [...],
    // references: [...],
    // summary: [...],
    // translation: [...],

    // ── Develop specific content ────────────────────────────────
    assistance-coding: [I asked for explanations about a Python code
      snippet and for help correcting an error.],
    // generating-content: [...],
    // optimization: [...],
    // data-processing: [...],
    // idea-inspiration: [...],
    // other: [...],
  ),
  usage-reflection: [
    Overall, I found generative AI useful for boilerplate tasks and
    quick explanations, but it required careful verification of all
    generated content. It did not replace critical thinking or
    domain-specific knowledge.
  ],
```

```
),
```

1.7.2. Option B: Provide custom content

If you need full control, pass any `content` directly:

```typ
genai-declaration: [
  // Your custom AI declaration content here.
],
```

1.8. Bibliography

Pass the result of Typst's built-in `bibliography()` call:

```typ
// config/bibliography.typ
#let bibliography-file = "../references.bib"
#let bibliography-style = "ieee"
#let bibliography = bibliography(bibliography-file, style: bibliography-style)
```

```typ
// report.typ
#import "config/bibliography.typ": bibliography

#show: conf.with(
  // ...
  bibliography-content: bibliography,
)
```

---

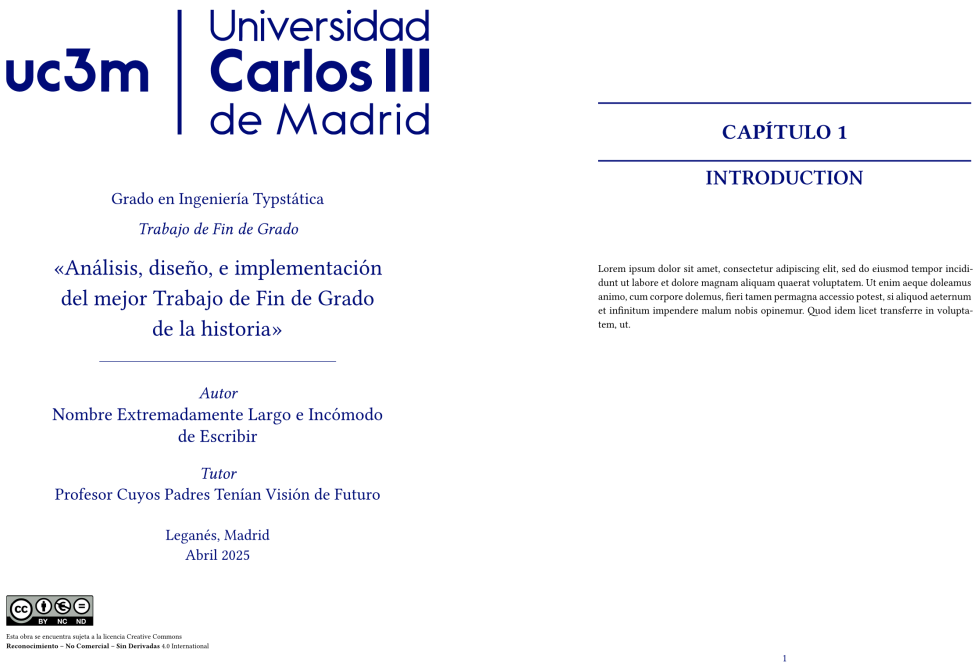
2. Styles

The template offers three visual styles, controlled by the `style` parameter.

2.1. `"fancy"` (default)

An opinionated style inspired by the original LaTeX template, with:

- Decorative chapter title pages with a horizontal rule and chapter number/name centered
- Serif headers showing the current chapter name and page number
- IEEE-style figure captions and table captions
- **Libertinus Serif** body font


Grado en Ingeniería Typstática

Trabajo de Fin de Grado

«Análisis, diseño, e implementación del mejor Trabajo de Fin de Grado de la historia»

Autor
Nombre Extremadamente Largo e Incómodo de Escribir

Tutor
Profesor Cuyos Padres Tenían Visión de Futuro

Leganés, Madrid
Abril 2025

Esta obra se encuentra sujeta a la licencia Creative Commons
Reconocimiento – No Comercial – Sin Derivadas 4.0 International

CAPÍTULO 1

INTRODUCTION

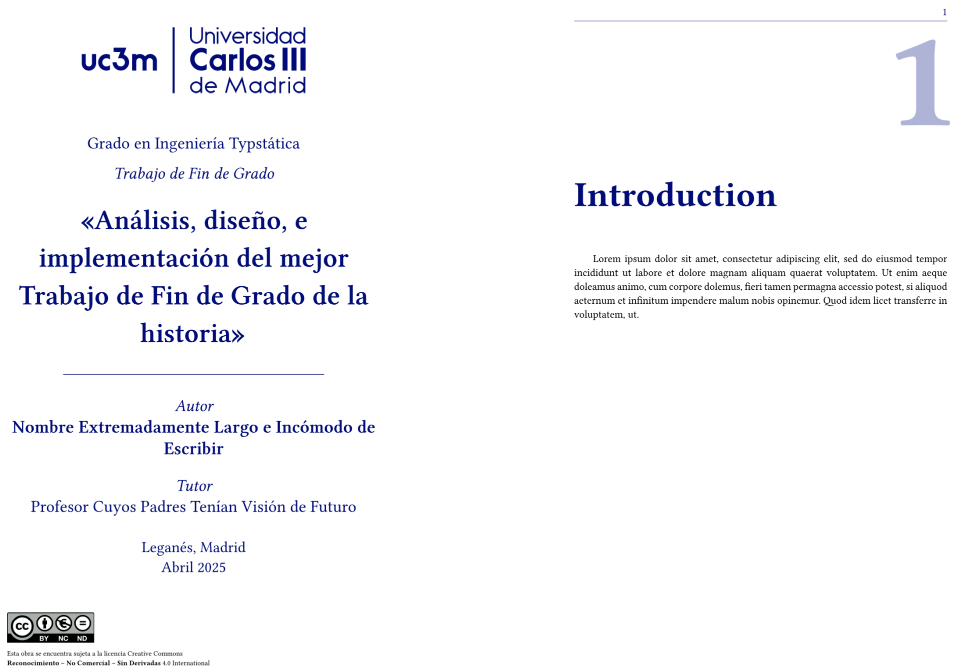
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1

Figure 1: Example of the **fancy** style.

2.2. `"clean"`

A clean, minimal style inspired by clean-dhbw, with:
- Large chapter number in the background on chapter pages
- Simple running header showing the chapter name
- Clean-style figure and table captions
- **Libertinus Serif** body font


Grado en Ingeniería Typstática

Trabajo de Fin de Grado

«Análisis, diseño, e implementación del mejor Trabajo de Fin de Grado de la historia»

Autor
Nombre Extremadamente Largo e Incómodo de Escribir

Tutor
Profesor Cuyos Padres Tenían Visión de Futuro

Leganés, Madrid
Abril 2025

Esta obra se encuentra sujeta a la licencia Creative Commons
Reconocimiento – No Comercial – Sin Derivadas 4.0 International

1

1

Introduction

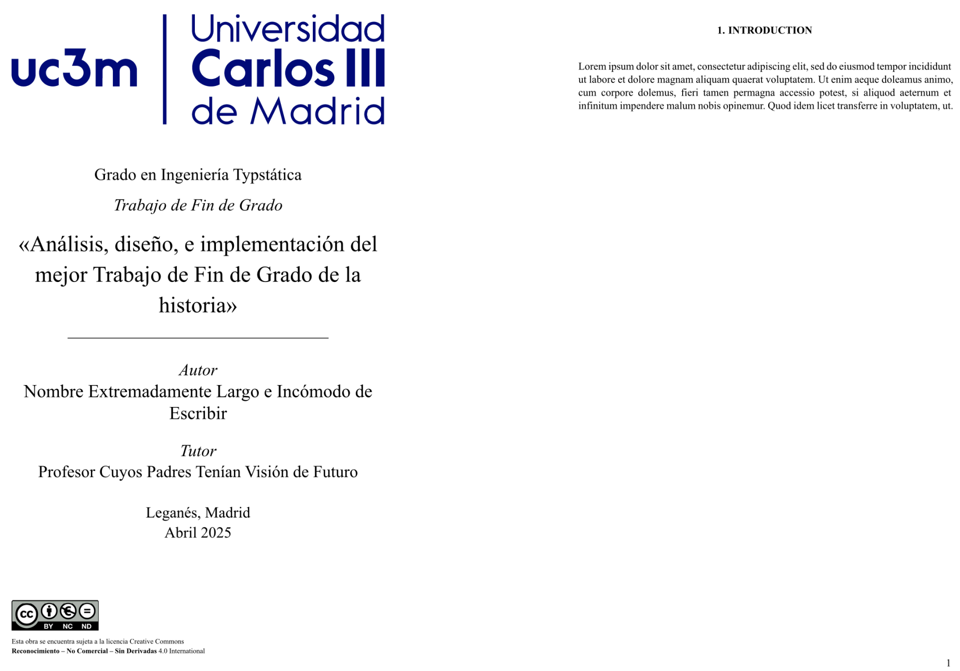
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Figure 2: Example of the **clean** style.

2.3. `"strict"`

Strictly follows the UC3M library guidelines:

- Times New Roman font, 12pt
- Centered uppercase chapter headings
- Page numbers in the footer (Roman numerals in front matter, Arabic in body)
- No decorative elements
- Double-sided layout is **not** permitted with this style


Grado en Ingeniería Typstática

Trabajo de Fin de Grado

«Análisis, diseño, e implementación del mejor Trabajo de Fin de Grado de la historia»

Autor
Nombre Extremadamente Largo e Incómodo de Escribir

Tutor
Profesor Cuyos Padres Tenían Visión de Futuro

Leganés, Madrid
Abril 2025

Esta obra se encuentra sujeta a la licencia Creative Commons
Reconocimiento – No Comercial – Sin Derivadas 4.0 International

1. INTRODUCTION

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1

Figure 3: Example of the **strict** style.

> [!NOTE] > You can mix styles for the title page and body independently using `titlepage-style`. For example, to use a `fancy` title page with a `strict` body: `style: "strict", titlepage-style: "fancy"`.

2.4. All parameters (API Reference)

lib

- conf()
- outline-entry-formatter()

conf

Main configuration function.

Recommended to use with `#show: conf.with(...)`.

Parameters

```
conf(
  doc: content,
  title: str,
  author: str,
  degree: str,
  advisors: array,
  location: str,
  thesis-type: str,
  date: datetime,
  bibliography-content: content,
  language: str,
  style: str,
  titlepage-style: str auto,
  table-style: str auto,
  figure-style: str auto,
  figure-spacing: length none,
  double-sided: bool,
  logo: str,
  short-title: str,
  date-format: str auto,
  license: bool,
  flyleaf: bool,
  epigraph: dictionary none,
  abstract: dictionary,
  english-abstract: dictionary,
  acknowledgements: content none,
  outlines: dictionaty none,
  appendixes: content none,
  glossary: array content none,
  abbreviations: dictionary content none,
  genai-declaration: dictionary content
) -> content
```

doc `content`

Thesis contents.

title `str`

Title of the thesis.

Default: none

author `str`

Author name.

Default: `none`

degree `str`

Degree name (e.g. `"Computer Science and Engineering"`).

Default: `none`

advisors `array`

List of advisor names.

Default: `none`

location `str`

Presentation location.

Default: `none`

thesis-type `str`

Type of thesis (`"TFG"` or `"TFM"`).

Default: `none`

date `datetime`

Presentation date.

Default: `none`

bibliography-content `content`

Bibliography contents, usually calling `bibliography`.

Default: `none`

language `str`

`"en"` or `"es"`.

Default: `none`

style `str`

Visual style, mainly affecting headings, headers, and footers. The available styles are `strict`, which strictly follow's the university library's guidelines, `clean`, based on clean-dhbw, and `fancy`, based on my original LaTeX version.

Default: `"fancy"`

titlepage-style `str` or `auto`

Style for the titlepage (see `style`). If set to `auto`, uses the main style.

Default: `auto`

table-style `str` or `auto`

Style for the table caption, either `"clean"` or `"ieee"`. If set to `auto`, uses the default for the main style (`"clean"` for `clean` style, `"ieee"` for the rest).

Default: `auto`

figure-style `str` or `auto`

Style for the figure caption, either `"clean"` or `"ieee"`. If set to `auto`, uses the default for the main style (`"clean"` for `clean` style, `"ieee"` for the rest).

Default: `auto`

figure-spacing `length` or `none`

Extra spacing to give to figures and tables. If `none`, no extra spacing.

Default: `0.75em`

double-sided `bool`

Whether to use double-sided pages. This is not allowed in the `strict` style.

Default: `false`

logo `str`

Type of logo (`"old"` or `"new"`).

Default: `"new"`

short-title `str`

Shorter version of the title, to be displayed in the headers. Only applies if `double-sided` is set to `true`.

Default: `none`

date-format `str` or `auto`

Date format. Use `auto` or specify the format using the [Typst format syntax](https://typst.app/docs/reference/foundations/datetime/#format).

Default: `auto`

license `bool`

Whether to include the CC BY-NC-ND 4.0 license.

Default: `true`

flyleaf `bool`

Whether to include a blank page after the cover.

Default: `true`

epigraph `dictionary` or `none`

A short quote that guided you through the writting of the thesis, your degree, or your life. Consists of `quote` (of type `content`), the body or text itself, `author` (of type `str`), the author of the quote and, optionally, `source` (of type `str`), where the quote was found.

Default: `none`

abstract `dictionary`

A short and precise representation of the thesis content. Consists of `body` (of type `content`), the main text, and `keywords`, an array of key terms (of type `str`) (see [IEEE Taxonomy](https://www.ieee.org/content/dam/ieee-org/ieee/web/org/pubs/ieee-taxonomy.pdf)).

Default: `none`

english-abstract `dictionary`

An english translation of the abstract. Compulsory for spanish works, invalid for english ones.

Default: `none`

acknowledgements `content` or `none`

Text where you give thanks to everyone that helped you.

Default: `none`

outlines `dictionaty` or `none`

Set of extra outlines to include (`figures`, `tables`, `listings`), and extra custom outlines (`custom`, an array of `contents` – typically the result of calling `outline`).

Default: `none`

appendixes `content` or `none`

Set of appendixes.

Default: `none`

glossary `array` or `content` or `none`

Glossary entries. If `array` is provided, it will use the `glossarium` library. If content is passed, it will display that content, without applying any styling.

Default: `none`

abbreviations `dictionary` or `content` or `none`

Abbreviations, acronyms and initials used throughout the thesis. You can provide a map (dictionary of strings) or a custom one (`content`).

Default: `none`

genai-declaration `dictionary` or `content`

Information about the use of Generative AI in the thesis. You can suply your own `content`, or use the university's template, by suplying a `dictionary`. See the example for more details.

Default: `none`

outline-entry-formatter

Formats an outline entry.

Parameters

```
outline-entry-formatter(
  it: content,
  prefix: auto content,
  body: auto content,
  fill: boolean,
  page: boolean,
  above: auto fraction relative,
  spacing: auto fraction relative,
  text-size: auto lenght,
  text-weight: int str,
  color: color,
  indented: boolean,
  justified: boolean
) -> content
```

it `content`

Outline entry.

prefix `auto` or `content`

Entry prefix (e.g. `[Chapter 1]`). If `auto`, default prefix.

Default: `auto`

body `auto` or `content`

Entry body (e.g. `[My chapter]`). If `auto`, default body.

Default: `auto`

fill `boolean`

When `true`, add dots between the body and the page number.

Default: `true`

page `boolean`

When `true`, shows the page number.

Default: `true`

above `auto` or `fraction` or `relative`

The spacing between this block and its predecessor (`block.above`).

Default: `auto`

spacing `auto` or `fraction` or `relative`

The spacing around the block (`block.spacing`). When `auto`, inherits the paragraph spacing.

Default: `auto`

text-size `auto` or `lenght`

Text size. When `auto`, default text size. (`text.size`)

Default: `auto`

text-weight `int` or `str`

Text weight (`text.weight`).

Default: `"regular"`

color `color`

Entry text color. Includes link color.

Default: `black`

indented `boolean`

Whether to indent the entry.

Default: `true`

justified `boolean`

When `false`, wraps the entry in a `block` to prevent justification.

Default: `true`

generative-ai

- genai-template()

genai-template

UC3M template for declaration of use of AI in the project.

Parameters

```
genai-template(
  language: str,
  usage: bool,
  data-usage: dictionary,
  technical-usage: dictionary,
  usage-reflection: content
)
```

language `str`

The language of the project (e.g. “en”, “es”).

usage `bool`

Whether AI is used in the project.

data-usage `dictionary`

AI data usage declaration.

technical-usage `dictionary`

Technical usage of AI.

usage-reflection `content`

Reflection on the usage of AI.